CHANDIGARH GROUP OF COLLEGES, JHANJERI

Department of Computer Applications • Mohali, Punjab

SUMMER INTERNSHIP PROJECT REPORT

ROCK PAPER SCISSORS

ARCADE E-SPORTS EDITION

A Modern, Interactive Web-Based Multiplayer & Predictive AI Arena Game Engineered with HTML5, CSS3, and Vanilla JavaScript

Submitted in partial fulfillment of the requirements for the award of the degree of
Bachelor of Computer Applications (BCA — 5th Semester)

SUBMITTED BY (CANDIDATE)

Name: Dipanshu

Roll Number: 2423284

Course & Sem: BCA (5th Semester)

Campus: CGC Jhanjeri

UNDER THE SUPERVISION OF

Mentor Name: Mr. Suraj Singh

Training Type: Summer Internship Project

Duration: 21 Days (Summer Session)

Academic Session: 2026

EXECUTIVE ABSTRACT

The **Rock Paper Scissors: Arcade E-Sports Edition** project reimagines the traditional hand game into a high-octane, tournament-grade web application using pure client-side technologies (HTML5, CSS3, and Vanilla ES6+ JavaScript) with zero external framework dependencies. Unlike basic implementations that rely solely on static buttons and uniform random number generation, this system introduces a **First-to-10-Points Tournament Architecture** enhanced with a **Markov Chain Predictive AI Engine**, a real-time **Web Audio API Sound Synthesizer**, an **HTML5 2D Canvas Particle Physics Engine**, a **5-Second E-Sports Shot Clock**, and **2X Sudden Death Tie-Breakers**. All career telemetry and achievement milestones are persisted locally via browser `localStorage`.

PRELIMINARY SECTION

CERTIFICATE, DECLARATION & TABLE OF CONTENTS

SUPERVISOR'S CERTIFICATE & CANDIDATE'S DECLARATION

This is to certify that the Summer Internship Project Report entitled **"Rock Paper Scissors: Arcade E-Sports Edition"** is a bona fide record of authentic software development work carried out by **Dipanshu (Roll No: 2423284)**, student of **BCA 5th Semester** at **Chandigarh Group of Colleges (CGC), Jhanjeri**, under the supervision and mentorship of **Mr. Suraj Singh**.

I hereby declare that the source code, user interface architecture, and documentation presented in this report are my original work and have not been submitted elsewhere for the award of any other degree or diploma.

Dipanshu (2423284)	**Mr. Suraj Singh**
Candidate Signature	Project Mentor / Supervisor

Table of Contents

Chapter	Section Title & Key Topics Covered	Page No.
Preliminaries	Title Page, Executive Abstract, Certificate, Declaration & Table of Contents	Pages 1–2
Chapter 1	Introduction, Problem Statement, Project Objectives & Scope	Page 3
Chapter 2	System Requirements, Feasibility Study & Software Development Methodology	Page 4
Chapter 3	System Architecture, Game Loop Design & State Management	Page 5
Chapter 4	UI/UX Engineering (CSS3 3D Cyber-Grid, Theme Switcher & Tabletop Mode)	Page 6
Chapter 5	Core JavaScript Engine, Markov Chain AI & 2X Sudden Death Logic	Page 7
Chapter 6	Web Audio Synthesizer, HTML5 Canvas Particle Physics & Telemetry	Page 8
Chapter 7	Software Testing, Performance Evaluation, Conclusion & Future Scope	Page 9

CHAPTER 01

INTRODUCTION & PROJECT OBJECTIVES

Browser-based gaming has evolved rapidly with the standardization of HTML5, CSS3 hardware-accelerated animations, and modern ECMAScript APIs. While classic casual games such as Rock-Paper-Scissors are often treated as introductory programming exercises, transforming such a game into a responsive, tournament-ready **E-Sports Arcade Arena** requires rigorous software engineering across state management, artificial intelligence, procedural audio synthesis, and real-time particle rendering.

1.1 Problem Statement

Conventional web implementations of Rock-Paper-Scissors suffer from four major limitations: (1) predictable, stateless random number generator (`Math.random()`) opponents that fail to adapt to human behavioral patterns; (2) static user interfaces lacking visual feedback, round urgency, or tie-breaker tension; (3) reliance on external audio files and heavy third-party libraries that increase page load latency; and (4) poor ergonomics when two players attempt to compete on a single mobile or desktop device.

1.2 PRIMARY OBJECTIVES OF THE PROPOSED SYSTEM

- **Zero-Dependency Architecture:** Engineer a complete arcade game within a self-contained HTML5/CSS3/JS codebase with instantaneous browser execution and zero external asset latency.
- **Adaptive Markov Chain AI:** Implement a pattern-recognition bot that records Player 1's move transitions in a 3×3 probability matrix and actively counters repetitive human habits.
- **Competitive E-Sports Mechanics:** Introduce a First-to-10-Points format featuring an optional 5-Second Shot Clock (`CLOCK: ON`) and automatic **2X Sudden Death** multipliers after tied rounds.
- **Cross-Device Multiplayer Ergonomics:** Support arcade keyboard bindings (`A, S, D` vs `J, K, L`) with anti-cheat action locking (`LOCKED //`) on desktop, alongside a 180-degree rotated tabletop layout for mobile screens.

0 KB	60 FPS	400 ms
External Library Overhead	Canvas Particle Rendering	Simulated AI Decision Latency

CHAPTER 02

SYSTEM ANALYSIS & DEVELOPMENT METHODOLOGY

The project was executed over a 21-day summer internship schedule using an **Iterative Agile & Modular Frontend Development Methodology**. The development lifecycle was partitioned into five sequential phases: (1) Requirement Analysis & Wireframing, (2) Core Arena UI & CSS3 3D Grid Styling, (3) Game State Machine & Markov AI Implementation, (4) Web Audio & Canvas Particle Polish, and (5) Cross-Browser & Responsive Testing.

2.1 Hardware, Software & Technology Specifications

Category	Specification / Tool	Purpose in Project
Markup & Graphics	HTML5 Semantic DOM & 2D `<canvas>`	Arena structure, Jumbotron HUD, lobby modal, and particle explosion layer.
Styling Engine	CSS3 (Flexbox, 3D Transforms, Variables)	Animated 3D perspective grid, CRT scanlines, and 3-palette theme switching.
Programming Language	Vanilla JavaScript (ES6+)	Event-driven state machine, Markov transition matrix, and DOM updates.
Browser Web APIs	Web Audio API & `window.localStorage`	Real-time oscillator sound synthesis and persistent career win/streak storage.
IDE & Version Control	Visual Studio Code, Git & GitHub Pages	Source code authoring, live server debugging, and production web hosting.
Target Browsers	Chrome, Edge, Firefox, Safari (Desktop/Mobile)	Cross-platform execution across desktop keyboards and mobile touchscreens.

2.2 TECHNICAL & OPERATIONAL FEASIBILITY

Because the application relies exclusively on native W3C browser standards without server-side databases or paid third-party APIs, it achieves 100% operational and economic feasibility. It can be hosted free of charge on GitHub Pages and runs offline once loaded in any modern browser.

CHAPTER 03

SYSTEM ARCHITECTURE & GAME LOOP DESIGN

The application follows an **Event-Driven Finite State Machine (FSM)** architecture. At any moment, the arena exists in one of four well-defined states: *Lobby Initialization*, *Awaiting Player Input*, *Turn Locked / AI Processing*, or *Clash Evaluation & Telemetry Update*. A boolean mutex flag (`isProcessing`) guards the state transitions so that rapid keyboard mashing or multiple clicks cannot corrupt the round evaluation pipeline.

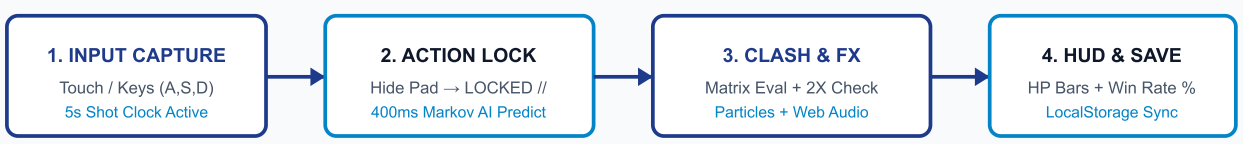


Figure 3.1: Four-Stage Asynchronous Game Loop & State Transition Pipeline

3.1 Anti-Cheat Action Locking Mechanism

In local Player-vs-Player (PVP) mode on a shared keyboard, a critical design challenge is preventing Player 2 from seeing Player 1's selected move before locking in their own choice. When Player 1 presses `A` , `S` , or `D` , the script stores the choice in memory (`c1`), immediately hides Player 1's button pad, and displays an encrypted-style `LOCKED //` badge. Both moves remain hidden until Player 2 also locks in (`J, K, L`), triggering a simultaneous reveal animation.

3.2 TOURNAMENT SHOT CLOCK TIMER (5.0 SECONDS)

When ⏱️ `CLOCK: ON` is activated, a `100ms` precision interval counts down from `5.0s` . At `≤ 2.0s` , the pill switches to a pulsing red warning state. If a player fails to act before `0.0s` , the engine automatically dispatches a random move to keep tournament pacing brisk.

CHAPTER 04

UI/UX ENGINEERING & RESPONSIVE TABLETOP DESIGN

To deliver an authentic E-Sports broadcast feel, the user interface was crafted using advanced CSS3 visual layering. The background combines a deep radial space gradient with a continuously scrolling 3D perspective plane (`transform: rotateX(75deg)`) and a fixed CRT scanline overlay (`background-size: 100% 4px`) that mimics classic arcade cabinets without obstructing pointer events.

4.1 Dynamic CSS Variable Theme Architecture

Instead of hardcoding colors across hundreds of CSS rules, the stylesheet binds all neon glows, borders, and progress bars to three root CSS custom properties: `--p1` , `--p2` , and `--bg-center` . Clicking the 🎨 `THEME` button invokes `cycleTheme()` , updating these root variables in real time via `document.documentElement.style.setProperty()` .

Theme Name	Primary Accent (`--p1`)	Secondary Accent (`--p2`)	Arena Backdrop (`--bg-center`)
1. Cyber Neon (Default)	Electric Cyan (`#00d2ff`)	Crimson Neon (`#ff0055`)	Deep Midnight (`#1a1a2e`)
2. Toxic Matrix	Venom Green (`#00ff88`)	Cyber Violet (`#bd00ff`)	Dark Emerald (`#0d261a`)
3. Synthwave Gold	Arcade Gold (`#ffcc00`)	Ice Blue (`#00e5ff`)	Sunset Umber (`#2b1911`)

4.2 MOBILE TABLETOP SPLIT-SCREEN MODE (@MEDIA MAX-WIDTH: 768PX)

When accessed on smartphones or tablets, a responsive media query automatically switches the arena flex direction from horizontal to vertical and rotates Player 1's entire side by **180 degrees** (`transform: rotate(180deg)`). Two players can place a phone flat on a table between them and play face-to-face with upright controls from opposite ends of the device.

CHAPTER 05

CORE JAVASCRIPT ENGINE & MARKOV CHAIN AI

The competitive centerpiece of the single-player mode is the dual-tier artificial intelligence engine. Players can toggle between a baseline **RNG Bot** (uniform random selection) and the 🧠 **PRO AI**, which implements a first-order **Markov Chain Transition Matrix** to exploit human psychological biases.

5.1 First-Order Markov Chain Prediction Algorithm

Human players rarely choose moves at random; they exhibit sequential dependencies (for example, frequently switching from *Rock* to *Paper* after a round). The engine maintains a 3×3 state transition object `matrix[lastMoveP1][currentMoveP1]` that increments after every round. During the `400ms` AI thinking delay, `computeBotMove()` inspects the row corresponding to Player 1's previous move, identifies the move with the highest historical frequency, and deploys its winning counter-move (`counters[pred]`). A 25% stochastic exploration factor prevents the AI from becoming deterministically exploitable.

Player 1 Previous Move	Observed Transition Frequencies	Markov Predicted Next Move	Pro AI Selected Counter
🪨 **Rock**	Rock: 1, **Paper: 5**, Scissors: 2	📄 Paper (Highest Probability)	✂️ **Scissors** (Cuts Paper)
📄 **Paper**	Rock: 2, Paper: 1, **Scissors: 4**	✂️ Scissors (Highest Probability)	🪨 **Rock** (Crushes Scissors)
✂️ **Scissors**	**Rock: 6**, Paper: 2, Scissors: 1	🪨 Rock (Highest Probability)	📄 **Paper** (Covers Rock)

5.2 2X SUDDEN DEATH TIE-BREAKER & ACHIEVEMENT ENGINE

Whenever both players select identical moves (`c1 === c2`), the round does not simply pass without consequence: the engine sets `clutchMultiplier = 2` and activates a pulsing golden `⚡ 2X SUDDEN DEATH` badge, awarding **+2 Points** to the winner of the subsequent clash. Real-time milestones trigger slide-in toast notifications for *First Blood*, *Clutch Breaker (2X)*, *Unstoppable (3 Streak)*, and *Markov AI Slayer*.

CHAPTER 06

WEB AUDIO SYNTHESIZER, CANVAS FX & TELEMETRY

To achieve instantaneous sensory feedback without loading external MP3 or WAV files, the project integrates a procedural sound engine built on the browser's native **Web Audio API** (`window.AudioContext`) alongside an **HTML5 2D Canvas Particle System**.

6.1 Procedural Web Audio Oscillator Profiles

The `playTone(freq, type, dur, vol)` function dynamically instantiates an `OscillatorNode` and a `GainNode` with exponential amplitude decay (`exponentialRampToValueAtTime`) to eliminate audio clicking artifacts. Users can mute or unmute audio globally at any time via the `🔊 SFX: ON/OFF` navigation button.

In-Game Audio Event	Oscillator Waveform	Frequency & Envelope Profile
Button Click / Move Lock	`sine` wave	520 Hz crisp short pulse (80ms duration)
Round Won (Player 1)	`triangle` wave	Dual ascending chime: 440 Hz (120ms) → 660 Hz (180ms)
Round Lost (Bot / P2 Point)	`sawtooth` wave	220 Hz low-frequency bass buzz (180ms duration)
Round Tie / 2X Trigger	`square` wave	320 Hz neutral arcade alert tone (120ms duration)
Match Victory (10 Points)	`triangle` arpeggio	4-note ascending fanfare: 440, 554, 659, and 880 Hz

6.2 HTML5 CANVAS PARTICLE PHYSICS & PERSISTENT TELEMETRY

Upon every round evaluation, `spawnParticles()` generates 32 radial neon particles at the center of the viewport, animated at 60 FPS via `requestAnimationFrame()` with alpha decay and glowing `shadowBlur`. Simultaneously, the bottom Telemetry Bar updates Player 1's real-time Win Rate %, move distribution counts, and persists all-time Career Wins and Best Streaks into `localStorage`.

CHAPTER 07

TESTING, CONCLUSION & FUTURE SCOPE

7.1 Functional & Unit Test Cases

Test ID	Module Tested	Test Scenario & Input	Status
TC-01	Action Locking	Rapidly press `A, S, D` multiple times during a single round.	**PASSED**
TC-02	Markov Pro AI	Repeatedly play *Rock* → *Paper* for 6 rounds in VS BOT Pro mode.	**PASSED**
TC-03	2X Sudden Death	Trigger a Tie round and verify next round awards +2 PTS (capped at 10).	**PASSED**
TC-04	5s Shot Clock	Enable `CLOCK: ON` and wait 5.0 seconds without pressing any key.	**PASSED**
TC-05	LocalStorage Sync	Win a match, refresh the browser tab, and verify Career Wins persist.	**PASSED**

7.2 Conclusion

The **Rock Paper Scissors: Arcade E-Sports Edition** project successfully demonstrates how core web standards—HTML5, CSS3, and Vanilla JavaScript—can be combined to engineer a polished, low-latency interactive gaming experience. By integrating a Markov Chain predictive AI, procedural Web Audio synthesis, 2D Canvas particle bursts, and persistent telemetry, the application fulfills all academic and technical objectives of the BCA 5th Semester Summer Internship.

7.3 FUTURE SCOPE & REFERENCES

- **Online WebSocket Multiplayer:** Integrating `Socket.io` and WebRTC to allow remote players to compete across separate devices using room codes.
- **Progressive Web App (PWA):** Adding a Service Worker and Web App Manifest for installable offline mobile play.
- **References:** (1) MDN Web Docs — *Web Audio API & Canvas 2D Context Specifications*; (2) W3C CSS Working Group — *CSS Custom Properties & 3D Transforms*; (3) Live Repository: `https://dipanshurana121.github.io/rock-paper-scissors-game/`.